

7.2 实验二：航线推送与航点解析文件

7.2.1 实验目的

本实验旨在掌握 QGC WPL 110 格式的航点文件结构和含义；掌握如何在 ROS 环境中解析航点文件，完成航点序列的读取、结构化存储与下发；学会调用 MAVROS 中的/mavros/mission/push 服务，实现向飞控推送航点，初步理解自动化流程中语音识别触发任务的实现逻辑。

7.2.2 实验说明

地面站生成的航线通常以标准化航点文件形式保存。本实验以 QGC 航点文件格式为例，说明航点字段含义与解析流程，并通过 MAVROS 的任务推送服务将航点写入飞控任务列表，实现“文件-解析-下发”的闭环。QGroundControl 导出的航线文件为 QGC WPL 110 格式的纯文本文件。每一行表示一个航点，其格式为：index current frame command param1 param2 param3 param4 x_lat y_long z_alt autocontinue。此顺序和 mavros_msgs::Waypoint 消息的字段顺序是不同的。解析完成后，通过调用 ROS 服务 /mavros/mission/push 将航点以 mavros_msgs::Waypoint 的形式推送给飞控，从而实现自动任务规划。

7.2.3 实验步骤

步骤一：新建工程包

在工作空间的 src 下面创建工程包，命令如下：catkin_create_pkg kadli rospy roscpp std_msgs roslib 其中 kadli 是包名，后面的都是依赖项列表。可在 CMakeLists.txt 文件中查看到添加的依赖项，如图 7.14 所示：

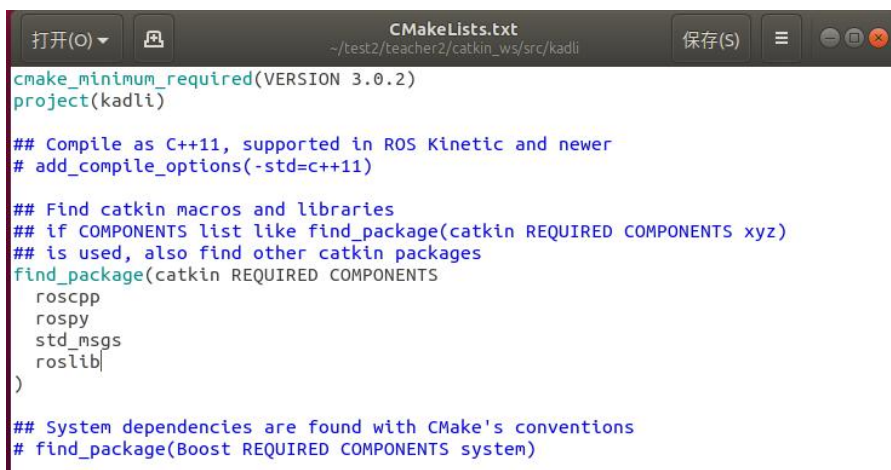


图 7.14 查看依赖项

步骤二：规划航线

使用 MissionPlanner 进行航线规划，将任务导出为 .waypoints 文件。导出的文件采用 QGC WPL 110 格式，该格式是 MissionPlanner 和 QGroundControl 等地面站通用的航点任务文本格式，适用于 ArduPilot 飞控系统。存放于路径 `~/test2/student2/catkin_ws/src/kadli/config`（在 kadli 包下面新建文件夹命名 config），并确保航线规划正确。航点文件如图 7.15 所示：

```

QGC WPL 110
0      1      0      16      0      0      0      0      31.772348      117.19
1      0      3      22      15.00000000      0.00000000      0.00000000      0.0000
2      0      3      16      0.00000000      0.00000000      0.00000000      0.0000
3      0      3      16      0.00000000      0.00000000      0.00000000      0.0000
4      0      3      16      0.00000000      0.00000000      0.00000000      0.0000
5      0      3      16      0.00000000      0.00000000      0.00000000      0.0000
6      0      3      16      0.00000000      0.00000000      0.00000000      0.0000
7      0      0      177      2.00000000      1.00000000      0.00000000      0.0000
8      0      3      16      0.00000000      0.00000000      0.00000000      0.0000
9      0      3      16      0.00000000      0.00000000      0.00000000      0.0000
10     0      3      16      0.00000000      0.00000000      0.00000000      0.0000
11     0      3      16      0.00000000      0.00000000      0.00000000      0.0000
12     0      3      16      0.00000000      0.00000000      0.00000000      0.0000
13     0      3      16      0.00000000      0.00000000      0.00000000      0.0000
14     0      3      16      0.00000000      0.00000000      0.00000000      0.0000
15     0      3      16      0.00000000      0.00000000      0.00000000      0.0000
16     0      3      16      0.00000000      0.00000000      0.00000000      0.0000
17     0      3      16      0.00000000      0.00000000      0.00000000      0.0000
18     0      3      21      5.00000000      0.00000000      0.00000000      1.0000
  
```

图 7.15 查看 .waypoint 文件

航点文件格式解析如表 7.1 所示：

表 7.1 QGC WPL 110 格式解析

ind	Curr	fra	com	Para	Para	Para	Para	x_lat	y_long	z_	autocon
ex	ent	me	mand	m1	m2	m3	m4			alt	timue
0	1	0	16	0	0	0	0	31.772	117.19	30	1
								2348	25207		
1	0	3	22	15	0	0	0	0	0	30	1
2	0	3	16	0	0	0	0	31.771	117.19	30	1
								5594	2322		
3	0	3	16	0	0	0	0	31.771	117.19	30	1
								6198	29747		
4	0	3	16	0	0	0	0	31.772	117.19	30	1
								3701	2929		

其中 index 表示航点编号，Current 表示是否是当前任务（1 是当前点，其他是 0），frame 是坐标系（3 表示相对高度），command 是 MAVLink 命令编号（16 表示 waypoint，22 表示 take_off），param1~param4 是命令的参数，x_lat 是纬度，

y_long 是经度，z_lat 是高度，autocontinue 表示是否继续执行下一条命令（1 表示是，0 表示否）。首行 QGC WPL 110 是文件标识和版本号，从第二行开始，每行代表一个航点，读取每行航点顺序也应该如表 7.1 所示。

步骤三：解析航点文件并上传服务推送航点

在 kadli 包的 src 下面新建 speech_recognition.cpp 文件，来解析航点文件。打开文件，跳过第一行格式标识符（QGC WPL 110）。按照上述表格字段顺序逐行读取与解析，使用 std::istringstream 将文本按列拆分，并依次赋值给对应的变量，最终转换为 mavros_msgs::Waypoint 消息结构，用于在 ROS 中发布航线任务。解析航点文件代码框架：

```
// 解析 QGC WPL 110 格式的航点文件
std::vector<mavros_msgs::Waypoint> parseWaypoints(const std::string& filepath)
{
    /***/
    //以下读取并解析航点文件程序由学生填写

    /***/
}
```

步骤四：在 ROS 中创建服务客户端并推送航点

在 main() 函数里面初始化 ROS 节点，创建一个 NodeHandle 实例用于后续发布任务。读取航点文件路径，使用上一步写好的函数 parseWaypoints() 解析航点文件，创建 MAVROS 提供的 /mavros/mission/push 服务客户端，构造 WaypointPush 请求，将航点列表填入请求信息。调用服务接口，将航线任务上传至飞控系统。

步骤五：验证功能

修改 ./bashrc 里面的环境变量，CMakeLists.txt 文件末尾增加可执行文件，添加依赖关系以及链接相应的库，如图 7.16 所示：

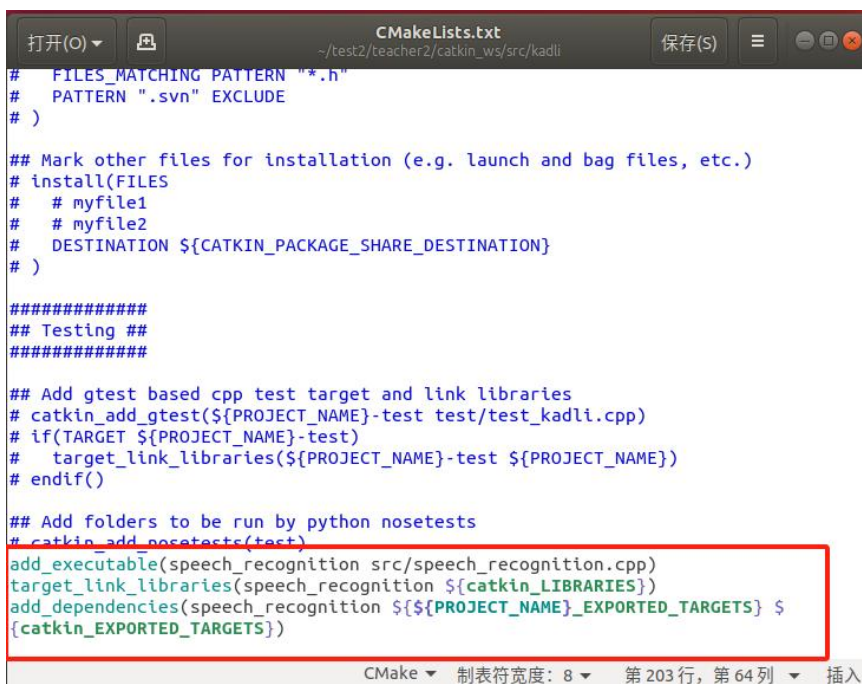


图 7.16 配置 CMakeLists.txt 文件

编译工作空间，运行，结果如图 7.17 所示：

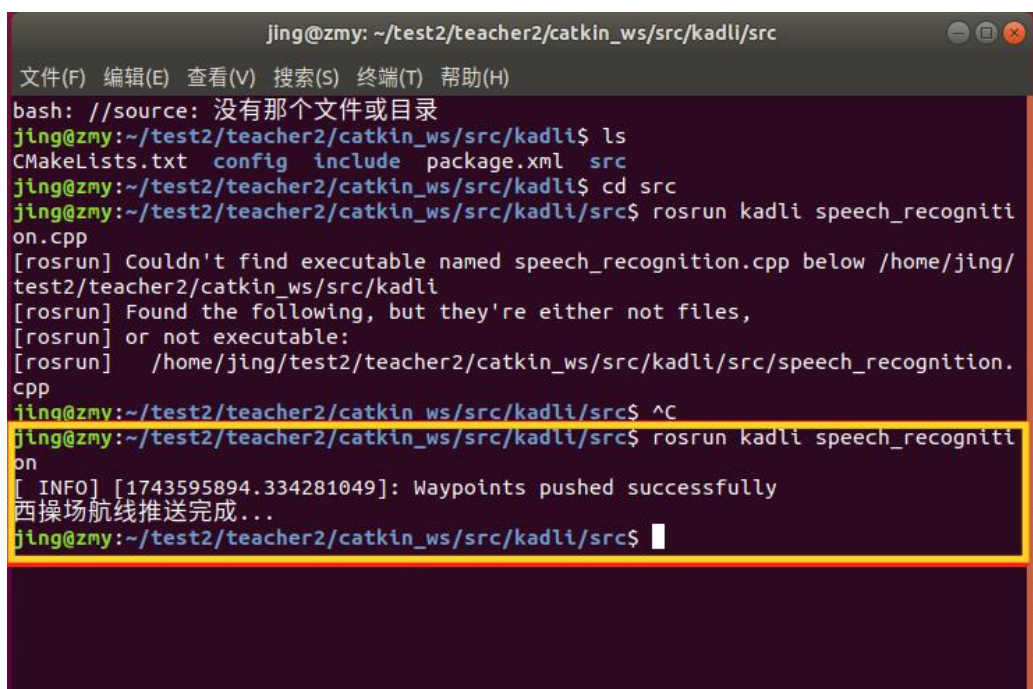


图 7.17 运行 speech_recognition 节点

打开 MissionPlanner，通常为飞控直接连接电脑波特率为 115200，数传连接波特率为 57600，选择接入的 COM 端口，在此采用的是数传连接。右上角连接，如图 7.18 所示：

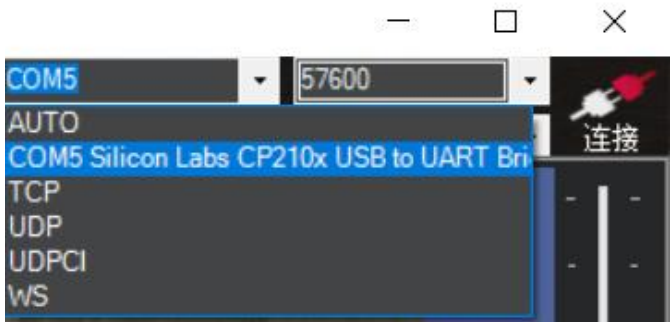


图 7.18 MissionPlanner 连接飞控

选择“飞行计划”，点击“读取航点”，即可看到飞控里面的航线，即我们推送过去的西操场航线，如图 7.19 所示：

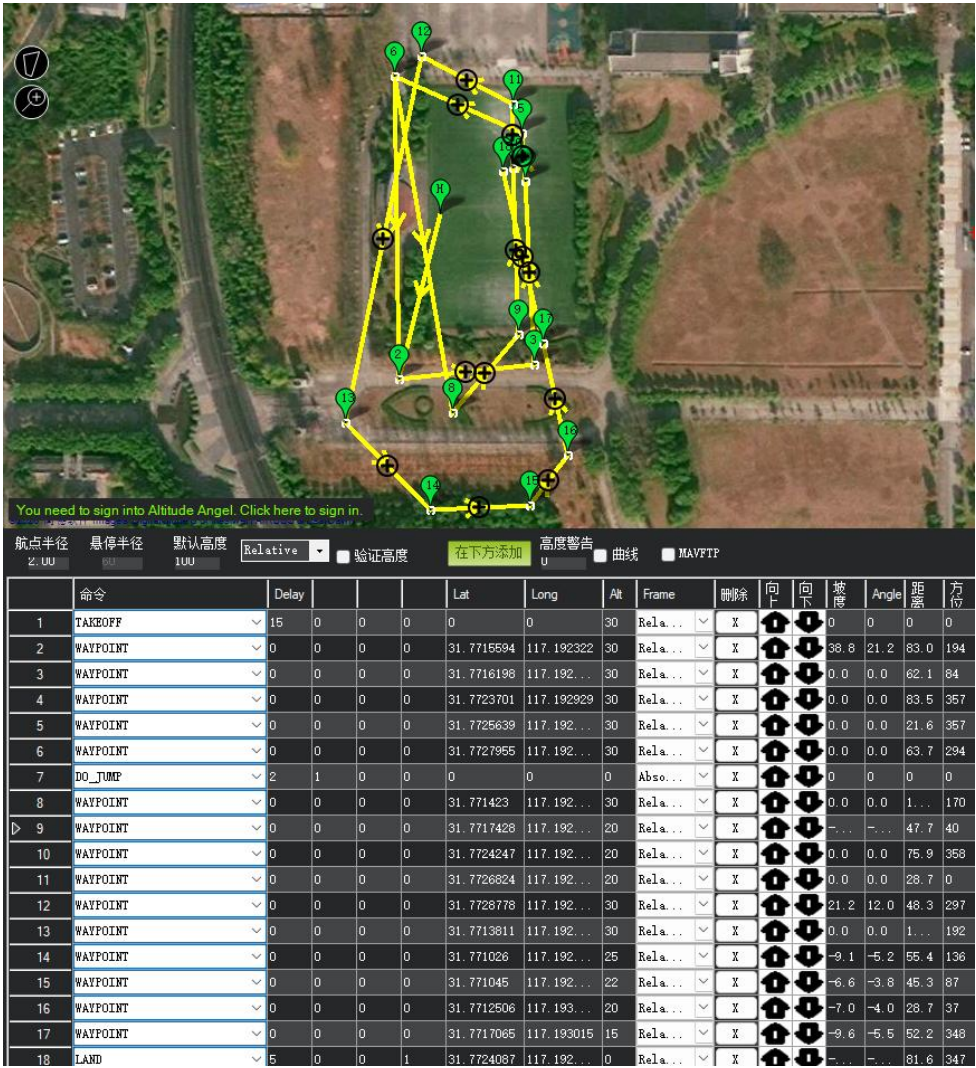


图 7.19 读取航点

参考代码：

```
// 解析 QGC WPL 110 格式的航点文件
std::vector<mavros_msgs::Waypoint> parseWaypoints(const std::string& filepath)
{
    std::vector<mavros_msgs::Waypoint> waypoints;
    std::ifstream file(filepath);
    std::string line;
    // 忽略文件头（第一行）
    std::getline(file, line);
    while (std::getline(file, line)) {
        std::istringstream iss(line);
        mavros_msgs::Waypoint wp;
        // 解析每一行的数据
        int seq, frame, command, current, autocontinue;
        double param1, param2, param3, param4, x_lat, y_long, z_alt;
        if (!(iss >> seq >> current >> frame >> command >> param1 >>
param2 >> param3 >> param4 >> x_lat >> y_long >> z_alt >> autocontinue)) {
            ROS_ERROR("Failed to parse waypoint line: %s", line.c_str());
            continue;
        }
        // 填充航点消息
        wp.frame = frame;
        wp.command = command;
        wp.is_current = (current == 1);
        wp.autocontinue = (autocontinue == 1);
        wp.param1 = param1;
        wp.param2 = param2;
        wp.param3 = param3;
        wp.param4 = param4;
        wp.x_lat = x_lat;
        wp.y_long = y_long;
        wp.z_alt = z_alt;
        waypoints.push_back(wp);
    }
    return waypoints;
}

//主函数
int main(int argc, char **argv)
{
    ros::init(argc, argv, "speech_recognition");
    ros::NodeHandle nh;
    //读取航点文件路径
    std::string package_path = ros::package::getPath("kadli");
    std::string waypoint_file = package_path +
"/config/xicaochang.waypoints";
```

```
        std::vector<mavros_msgs::Waypoint> waypoints =  
parseWaypoints(waypoint_file);  
        //创建 MAVROS 服务客户端  
        ros::ServiceClient wp_push_client =  
nh.serviceClient<mavros_msgs::WaypointPush>("/mavros/mission/push");  
        //准备服务请求  
        mavros_msgs::WaypointPush wp_push_srv;  
        wp_push_srv.request.waypoints = waypoints;  
        //调用服务  
        if(wp_push_client.call(wp_push_srv))  
        {  
            ROS_INFO("Waypoints pushed successfully");//成功推送时打印输出  
        }  
        else  
        {  
            ROS_ERROR("Failed to call /mavros/mission/push service");  
        }  
        std::cout << "西操场航线推送完成... " << std::endl;  
    }  
}
```